Chalmers | Göteborgs Universitet

Concurrent Programming TDA384/DIT392

22 August 2024

Exam supervisors: N. Piterman (piterman@chalmers.se, 031 772 6053) and G. Schneider (gersch@chalmers.se, 031 772 6073)

(Exam set by N. Piterman and G. Schneider, based on the course given in Aug-Oct 2023 and Jan-Mar 2024)

Material permitted during the exam (hjälpmedel):
Two textbooks; four sheets of A4 paper with notes (single or double-sided); English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60-79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; five questions, numbered Q1 through Q5. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

Q1 (15p). In the rest of this question we assume an implementation of semaphores that does not allow to go beyond the capacity of the semaphore. In this implementation, the capacity is different from the initialisation, that is, when you create a new semaphore, you need to give both the capacity and the initial value of the semaphore. A call to `up()` will *block* when the value of the internal counter of the semaphore is equal to the capacity (i.e., the thread will wait till the semaphore's counter is strictly less than the capacity, acting in the same way as `down()` does when the counter is zero). Note that, however, if the initial value of the semaphore's internal counter is higher than the capacity, you can still call `down()`. When you define a semaphore you will need to provide both the initial value and the capacity. The down operation (calling `sem.down()`) works as usual.

For instance, if you want to define a semaphore `sem` with capacity 5 and initial value 2, you will declare it as follows:

```
sem = new semaphore(2)[capacity=5]
```

In the example, the internal counter of `sem` is set originally to 2, so it is possible to make three consecutive calls to `sem.up()`, but it will *block* if a thread tries to call `sem.up()` again (assuming no other thread called `sem.down()` before).

In what follows you will get 5 questions concerning locks and semaphores. You need to justify your answer in each case (an answer without a justification will not be granted full points).

1 **(2p)** The pseudocode of the program in Fig. 1 shows the interaction between two threads operating on a shared variable `count`. We assume that there is no other thread using the semaphore. For this particular example the behaviour of the program does not change if we define the capacity of the semaphore to be 1. True or false? Justify.

Answer: True. The use of the semaphore is guaranteed to behave like a lock in this case, and the way up and down are called ensures the internal counter of the semaphore is always 0 or 1 (thus its capacity may well be set to 1).

2 **(2p)** Let us consider again the pseudocode of the program shown in Fig. 1 (as before, we assume that there is no other thread using the semaphore). Let us assume we now declare a lock. In that program, and under the above assumptions, we can safely replace the semaphore by a lock by **only** performing the following replacements: each `sem.up()` is replaced by `lock.unlock()` and

```
int count; Semaphore sem = new Semaphore(1)[capacity=2];
```

	thread t		thread u	
	`int a,b;`		`int c;`	
1	`sem.down();`		`sem.down();`	6
2	`a = count - 1;`		`c = count + 1;`	7
3	`b = a * 2;`		`count = c - 1;`	8
4	`count = a + b + 1;`		`sem.up();`	9
5	`sem.up();`			

Figure 1: Q1(1-3): Pseudocode of program `someCounting`.

each `sem.down()` by `lock.lock()`. Is that true or false? (By *safely* we mean that the behaviour of the program is preserved when you do the replacement.)

If you answer *true*, then explain why the replacement preserves the original behaviour. If you answer *false*, explain why this is not the case and give examples of wrong behaviours occurring because of the replacement (that highlights why the replacement is not correct).

Answer: True. The way the semaphore is defined and used in the original program, mimics exactly the use of a lock: the down operations can only be called by one of the threads at any time, and the up operations are only performed at the end (and only by the thread calling the down operation). The capacity in this case does not affect the behaviour as the internal counter of the semaphore always remain with the values 0 and 1.

3 **(3p)** Let us now consider the very same program shown in Fig. 1, but let us assume that there might be other threads running in parallel with threads `t` and `u` and that they may both call `sem.up()` and `sem.down()`. Under such assumption, is it true that the program satisfies the following properties: i) mutual exclusion; ii) deadlock freedom

If you answer *true*, then explain why this is the case. If you answer *false*, explain which properties are not satisfied and why (give an argument and at least one example that shows a violation of the property).

Answer: False. A third thread may: i) call `sem.down()` *before*

```
int count; Semaphore sem = new Semaphore(1)[capacity=2];
```

	thread t		thread u	
	`int b;`		`int c;`	
1	`sem.up();`		`sem.up();`	6
2	`count = count + 1;`		`c = count + 1;`	7
3	`b = 2;`		`count = c - 1;`	8
4	`count = count + b;`		`sem.down();`	9
5	`sem.down();`			

Figure 2: Q1(4-5): Pseudocode of program `someCounting2`.

threads `t` *and* `u` *without never calling* `sem.up()`*. In that case the internal counter of the semaphore is 0, in which case threads* `u` *cannot enter the critical section, thus getting into a deadlock. ii) call* `sem.up()` *and never call* `sem.down()` *so the counter is 2. Then, threads* `t` *and* `u` *can both execute* `sem.down()` *and enter into the critical section, violating mutual exclusion. (Other situations are also possible.)*

4 Let us now consider the pseudocode of the program shown in Fig. 2, and let us assume that there is no other thread using the semaphore. State whether the following assertions are true or false (justify your answer in both cases):

i. **(2p)** The program satisfies: i) deadlock freedom; ii) mutual exclusion.

Answer: True. Since the capacity limits the number of times we can call up and since both programs always call up at the beginning (and down at the end), the semaphore acts as a lock (and the value of the semaphore always remains between 1 and 2).

ii. **(2p)** Let us assume we declare a lock. We replace the semaphore by a lock by **only** performing the following replacements: each `sem.up()` is replaced by `lock.unlock()` and each `sem.down()` by `lock.lock()`. Is it true that the behavior of the program does not change?

Answer: False. The original program satisfies mutual exclusion and deadlock freedom, but the suggested replacement

would break mutual exclusion as both threads would be able to enter the critical section as the first operation they call is unlock.

5 Let us now consider again the program shown in Fig. 2, but let us assume that there might be other threads running in parallel with threads `t` and `u` and that may call both `sem.up()` and `sem.down()`. State whether the following assertions are true or false (justify your answer in both cases):

i. **(2p)** The program satisfies: i) deadlock freedom; ii) mutual exclusion.

Answer: False. A third thread may: i) call `sem.up()` *before threads* `t` *and* `u` *without never calling* `sem.down()`*. In that case the internal counter of the semaphore is 2, in which case threads* `u` *cannot enter the critical section, thus getting into a deadlock. ii) call* `sem.down()` *and never call* `sem.up()` *so the counter is 0. Then, threads* `t` *and* `u` *can both execute* `sem.up()` *and enter into the critical section, violating mutual exclusion. (Other situations are also possible.)*

ii. **(2p)** Let us still assume that other threads may use the semaphore, but that now we have declared a lock which may be used by all threads. We replace the semaphore by a lock in the codes of threads `t` and `u` by **only** performing the following replacements: each `sem.up()` is replaced by `lock.lock()` and each `sem.down()` by `lock.unlock()`. Is it true that the behaviour of the program in Fig. 2 is preserved?

Answer: False. If other threads can interfere and one of them call `lock.lock()` *threads* `t` *and* `u` *could deadlock.*

Q2 (14p). Malin is presenting her new painting collection and has invited some friends and relevant people from the artistic scene to an opening exhibition. Guests to the exhibition can arrive at any time without previous notice, but they are required to say goodbye to Malin before leaving (that is, they are not to leave unless Malin is available to greet them goodbye). Malin can go in and out of the exhibition at any time. We model this behaviour in Java pseudo-code using a monitor class, as explained below.

- Malin and her guests correspond to concurrent threads — one thread per person.
- Synchronisation occurs through an instance of monitor class Exhibit:

  ```
  monitor class Exhibit implements IExhibit
  ```

  The interface `IExhibit` is defined in Fig. 3:
- Malin and the guests share an instance of the class Exhibit. Malin keeps calling methods `enterMalin()` and `exitMalin()`, and the guests continuously call `arrive()` and `leave()` (one thread `Guest i` for each guest i ($1 \leq i \leq n$)) (see Fig. 4).
- Methods `enterMalin()`, `exitMalin()`, and `arrive()` are non-blocking; method `leave()` is blocking if Malin is currently **not** at the exhibition, and terminates as soon as Malin enters the party again.

The objective is to provide an implementation of a *monitor class* `Exhibit` with interface `IExhibit`, whose behaviour follows the above description. You should use the Java pseudo-code for monitors that we used in our lectures, or any similar notation that describes monitors and whose semantics is clear. NOTE: Assume a *signal and continue* signalling discipline.

(Part a). (3p)

Declare the *condition variables* and other *private attributes* that you need in class Exhibit. For each of them, explain them with a concise comment.

Answer:

```
// Number of guests at the exhibition waiting to leave:
 int leaving;
// is Malin at the exhibition?
 boolean malinIn;
// Condition variable: can guests leave?
 Condition canLeave = new Condition();
```

(Part b). (4p)

Write the implementation of methods `arrive()` and `leave()`.

Answer:

```
void arrive(int guest) {
   // nothing to do
 }

void leave(int guest) {
  leaving += 1;
  while (!malinIn)
    canLeave.wait();
  leaving-= 1;
}
```

(Part c). (4p)

Write the implementation of methods `enterMalin()` and `exitMalin()`.

Answer:

```
void enterMalin() {
   malinIn = true;
   for (int k = 0; k < leaving; k++)
      canLeave.signal();
}

void exitMalin() {
  malinIn = false;
}
```

(Part d). (3p)

Would you have to change your implementation if you were to use monitors following the *signal and wait* signalling discipline instead of *signal and continue*? Justify your answer.

Answer: The `while` *loops checking for a condition while waiting also work under signal and wait, since Malin sets* `malinIn` *to true before signalling any thread. However, there may be a problem with the integer* `leaving` *being changed by the signalled thread while Malin loops on its elements, which may result in skipping signalling some blocked threads. A simple solution is to get a copy of leaving local to thread* `malin` *in method* `enterMalin` *to avoid the interference.*

```
interface IExhibit
{
  void enterMalin();    // Malin is available at the exhibition
  void exitMalin();    // Malin leaves the exhibition (momentarily)

  void arrive(int guest);   // a guest arrives at the exhibition
  void leave(int guest);    // a guest leaves the exhibition
}
```

Figure 3: Q2: Pseudocode of interface `IExhibit`.

`IExhibit exhibit = new Exhibit();`

malin		guest i	
1	`while (true) {`	`while (true) {`	7
2	`  // Stay some time:`	`  // Arrive at exhibit.`	8
3	`  exhibit.enterMalin();`	`  // (i is guest's ID):`	9
4	`  // Leave some time:`	`  exhibit.arrive(i);`	10
5	`  exhibit.exitMalin();`	`  // Leave the exhibit.`	11
6	`}`	`  // (when possible):`	12
		`  exhibit.leave(i);`	13
		`}`	14

Figure 4: Q2: Pseudocode of threads of program `Exhibit`.

Q3 (15p). In this question, we give three implementations of a leader election protocol. From time to time threads try to become leaders by calling the *attempt* function. Independently, other threads call the follow function and get the thread ID of the leader. An implementation should satisfy the following. If someone is the leader for a certain interval of time then during that time no other thread can become a leader (mutual exclusion). A call to *follow* should return a leader value that is true to the time of the return (i.e., the leader or 0). Analyze which of the following implementations indeed creates such a leader election protocol. For each implementation, state which of the required properties hold (and explain why) and which do not hold (and explain why/give an example execution).

Here are the variables (and their types) that used by the three implementations. All three use the same constructor.

```
private Semaphore s = new Semaphore(1);
private Lock l = new ReentrantLock();
private AtomicLong expiry = new AtomicLong();

private long leader = 0;

public Leader() {
    expiry.set(System.currentTimeMillis());
}
```

(Part a). (5p) Here is an implementation attempt using a lock. Notice that the atomic long is used here as a normal long with only get and set and not with compare and set.

```
public void attempt1(long interval) {
    l.lock();
    try {
    if (expiry.get() >= System.currentTimeMillis())
        return;

    leader = Thread.currentThread().getId();
    expiry.set(System.currentTimeMillis() + interval);
    } finally {
        l.unlock();
    }
}

public long follow1() {
    if (expiry >= System.currentTimeMillis()) {
        return leader;
    }
    return 0;
}
```

Answer: The implementation satisfies mutual exclusion. No two threads can be leaders at the same time. This follows from synchronization of the attempt *function on the lock. In order for two threads to be leaders at the same time, one would have to lock after the other and then they would get the correct leadership end time and not become leader.*

However, as the follow1 *function is not using the lock, it can return stale values or 0. For example, an execution of* follow *could start during the time of one leader. The value of leader is read but not returned yet. It is then returned when another leader is active. Similarly, the check could happen when* `expiry<System.currentTimeMillis()` *but the return of 0 would happen later.*

(Part b). (5p)

Here is an implementation attempt using an atomic long. Notice the use of compare and set unlike the previous implementation.

```
public void attempt2(long interval) {
    long current = expiry.get();
    if (current < System.currentTimeMillis()) {
        if (expiry.compareAndSet(current,
                System.currentTimeMillis() + interval)) {
            leader = Thread.currentThread().getId();
        }
    }
}

public long follow2() {
    if (expiry.get() >= System.currentTimeMillis()) {
        return leader;
    }
    return 0;
}
```

Answer: The implementation does not satisfy mutual exclusion. The compare and set makes sure that at the time of becoming leader, no one else changed the expiry time of leadership. However, if after the compare and set, a thread is stopped until leadership has expired then the thread could write itself as leader during the interval of another thread.

As before, the follow2 *function is not using any synchroniation and can return stale values of 0.*

(Part c). (5p)

Here is an implementation using a semaphore.

```
public void attempt3(long interval) {
    long current = expiry.get();
    if (current < System.currentTimeMillis()) {
        s.release();
    }
    if (s.tryAcquire()) {
        leader = Thread.currentThread().getId();
        expiry.set(System.currentTimeMillis() + interval);
    }
}

public long follow3() {
    if (s.tryAcquire()) {
        s.release();
        return 0;
    } else {
        return leader;
    }
}
```

Answer: Multiple threads can call the release in attempt3*. For example, by having two threads checking if the variable* current *is smaller than the current time and both calling release bringing the semaphore to value 2 or more.*

Then, both threads can be leaders for the same time.

Once the value of the semaphore can be non-zero while there is a leader, follow3 *gives completely wrong information.*

Q4 (14p). In this question you are asked to modify the Readers-Writers board in Erlang that was taught in class to a Leader-Follower board.

Here is the original code presented in class:

```
% get read access to `Board'
begin_read(Board) ->
   Ref = make_ref(),
   Board ! {begin_read, self(), Ref},
   receive
     {ok_to_read, Ref} -> ok_to_read
   end.

% release read access to `Board'
end_read(Board) ->
   Ref = make_ref(),
   Board ! {end_read, self(), Ref}.

% get write access to `Board'
begin_write(Board) ->
   Ref = make_ref(),
   Board ! {begin_write, self(), Ref},
   receive
     {ok_to_write, Ref} -> ok_to_write
   end.

% release write access to `Board'
end_write(Board) ->
   Ref = make_ref(),
   Board ! {end_write, self(), Ref}.

% board with no readers and no writers
empty_board() ->
   receive
     % serve read request
     {begin_read, From, Ref} ->
        From ! {ok_to_read, Ref}, % notify reader
        readers_board(1);         % board has one reader
     % serve write request synchronously
     {begin_write, From, Ref} ->
        From ! {ok_to_write, Ref}, % notify writer
        receive                    % wait for writer to finish
          {end_write, _From, _Ref} ->
            empty_board()          % board is empty again
        end
   end.

% board with no readers (and no writers)
readers_board(0) -> empty_board();
```

```erlang
% board with `Readers' active readers (and no writers)
readers_board(Readers) ->
    receive
      % serve write request
      {begin_write, From, Ref} ->
        % wait until all `Readers' have finished
        [receive {end_read, _From, _Ref} -> end_read end
          || _ <- lists:seq(1, Readers)],
        From ! {ok_to_write, Ref}, % notify writer
        receive                    % wait for writer to finish
          {end_write, _From, _Ref} -> empty_board()
        end;                       % board is empty again
      % serve read request
      {begin_read, From, Ref} ->
        From ! {ok_to_read, Ref},  % notify reader
        readers_board(Readers+1);  % board has one more reader
      % serve end read
      {end_read, _From, _Ref} ->
        readers_board(Readers-1)   % board has one less reader
    end.
```

A Leader-Follower board works slightly differently. There can be only one leader inside the board at any given time. However, followers need to have a leader in order to be inside the board. The number of followers that can be in the board at a given time (when there is a leader in) is unbounded and they can go in and out freely (as long as there is a leader inside). Once the leader wants to go out, it needs to make sure that all followers have left (and it is OK to stop new ones from coming in) and only then leave the board, which then waits for the next leader.

Your task is to modify the given implementation to that of Leader-Follower. Just like in the readers-writers board, you may assume that leaders and followers follow the engage-disengage protocol. For example, only followers who completed successfully a call to begin_follow would call end_follow, and similarly for leaders.

No points will be deducted for syntax issues. Make sure that indentation and intention is clear.

(Part a). (1p)

Implement `begin_follow` and `end_follow`, which should be counterparts of `begin_read` and `end_read`.

(Part b). (3p) Implement `begin_lead` and `end_lead`, which should be counterparts of `begin_write` and `end_write`.

(Part c). (5p) Implement `empty_board`.

(Part d). (5p) Implement `lead_board`.

It is allowed to design the board as one (or more than two) functions. In such a solutions the 10 points will be allocated according to the functionality supported.

Answer: Here is one possible implementation.

```
% get read access to `Board'
begin_follow(Board) ->
    Ref = make_ref(),
    Board ! {begin_follow, self(), Ref},
    receive
      {ok_to_follow, Ref} -> ok_to_follow
    end.

% release read access to `Board'
end_follow(Board) ->
    Ref = make_ref(),
    Board ! {end_follow, self(), Ref}.

% get write access to `Board'
begin_lead(Board) ->
    Ref = make_ref(),
    Board ! {begin_lead, self(), Ref},
    receive
      {ok_to_lead, Ref} -> ok_to_lead
    end.

% release write access to `Board'
end_lead(Board) ->
    Ref = make_ref(),
    Board ! {end_lead, self(), Ref},
    receive
       { ok_to_end_lead, Ref } -> ok_to_end_lead
    end.
```

```
% board with no leaders and no followers
empty_board() ->
    receive
      % serve lead request synchronously
      {begin_lead, From, Ref} ->
          From ! {ok_to_lead, Ref}, % notify leader
    lead_board(0)
    end.

lead_board(0) ->
    receive
       { end_lead, From, Ref } ->
           From ! { ok_to_end_lead, Ref },
      empty_board();
       { begin_follow, From, Ref } ->
           From ! { ok_to_follow, Ref },
      lead_board(1)
    end;

lead_board(Readers) ->
    receive
       { end_lead, From, Ref } ->
           [ receive {end_follow, _From, _Ref} -> end_follow end
             || _ <- lists:seq(1,Readers)],
      From ! { ok_to_end_lead, Ref },
      empty_board();
       { begin_follow, From, Ref } ->
           From ! { ok_to_follow, Ref},
      lead_board(Readers+1);
       { end_follow, _From, _Ref } ->
           lead_board(Readers-1)
    end.
```

Q5 (12p). In what follows you have 3 sub-questions (Parts a to e) concerning different topics seen in the course. Each part a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point) for each wrong choice.

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will not get -5 but 0.

NOTE: For each sub-question, there is exactly one option that is correct.

(Part a). (3p)

To find the total number of combinations of size R from a set of size N, where R is less than or equal to N, we use the following *combination* formula:

$$\frac{N!}{R!\,(N-R)!}$$

Fig. 5 shows an Erlang parallel algorithm to compute the above formula.

What is the maximal number of processes running in parallel when executing `parallelcomb` with parameters $N = 4$ and $R = 3$?

a) 1

b) 3

c) 4

d) $ln(7)$ (ln is the natural logarithm)

e) 8 $(N + R + (N - R))$

f) 13 $((N * R) + (N - R))$

Answer: Option c). Reason: the program launches three processes, one for each call to the factorial function, which is a sequential function. So, there will be a maximum of four processes in parallel: three parallel processes + the parent process.

(Part b). (3p)

```
-module(parallel_combinations).
-export([parallelComb/2, start/0]).

parallelComb(N, R) when N >= R, R >= 0 ->
    Parent = self(),
    spawn(fun() -> Parent ! {n_fac, factorial(N)} end),
    spawn(fun() -> Parent ! {r_fac, factorial(R)} end),
    spawn(fun() -> Parent ! {nr_fac, factorial(N-R)} end),

    N_Fac = receive {n_fac, Value} -> Value end,
    R_Fac = receive {r_fac, Value} -> Value end,
    NR_Fac = receive {nr_fac, Value} -> Value end,

    N_Fac div (R_Fac * NR_Fac).

factorial(0) -> 1;
factorial(N) when N > 0 -> N * factorial(N - 1).
```

Figure 5: Q5 - Part a: Erlang program `parallelComb`.

According to Amdahl's law, if the fraction p of a program can be parallelised, then, the maximum speedup that can be achieved by n processes is $\frac{1}{(1-p)+\frac{p}{n}}$.

We want to apply Amdahl's law to the Erlang program from sub-question a) above (program shown in Fig. 5), to calculate the speedup produced by this parallel version with respect to a purely sequential version of the program.

Which one of the following assertions is true?

a) There is no speedup at all of the parallel version w.r.t. the purely sequential version.

b) There is a speedup of 3, given that there are three spawned processes.

c) The parallel version can run more than 10 times faster than the purely sequential one.

d) The question is meaningless since Amdahl's law is used to make a prediction about the maximum performance improvement that can be expected from parallelising a program, and not to compare two versions of the same program.

e) None of the above.

Answer: Option d). Reason: Amdahl's law is not used to compare programs but to estimate the speedup if we know which percentage of the program can be parallelised.

(Part c). (3p)

The monitor class below implements a strong semaphore:

```
monitor class StrongSemaphore implements Semaphore {
  int count;
  Condition isPositive = new Condition(); // is count > 0?

  public void down() {
    while (count > 0)
      count = count - 1;
    isPositive.wait();
  }


  public void up() {
    if (isPositive.isEmpty())
      count = count + 1;
    else isPositive.signal();
  }
}
```

a) Yes. Assuming that there are no spurious wakeups and interruptions.

b) Only under signalling policy signal and continue.

c) Only under singnalling policy singal and wait.

d) No.

Answer: Option d). Down reduces count to 0 and then waits. This is regardless of the initial value of count.

(Part d). (3p)

The monitor class below implements a barrier.

```java
public class BarrierMonitor {
    private int expect;
    private int current;

    public BarrierMonitor(int n) {
        this.expect = n;
        this.current = 0;
    }

    public synchronized void await() throws InterruptedException  {
        current++;
        if (current<expect) {
            wait();
        }
        else {
            current = 0;
            notifyAll();
        }
    }
```

a) Yes. Assuming that there are no spurious wakeups and interruptions.

b) Only under signalling policy signal and continue.

c) Only under singnalling policy singal and wait.

d) No.

Answer: Option a). The synchronization of the await method ensures that current correctly captures the number of arrivals. Once it reaches expected, all are notified and continue to the next round.